ICCPracyiva03.1

Pol Blanco Medina

October 2026

1 01 Referencias e inmutabilidad de String

Código de la práctica

```
public class Ejercicio1 {
    public static void main(String[] args) {
        String a = "Java";
        String b = new String("Java");
        String c = "Ja" + "va";
        String d = "Ja";
        d += "va";
        System.out.println(a == b);
        System.out.println(a == c);
        System.out.println(a == d);
        System.out.println(a.equals(d));
        b = b.concat(" 21");
        System.out.println(a);
        System.out.println(b);
    }
}
```

Indica qué imprime cada instrucción y qué variables comparten un objeto.

El programa imprime:

```
false
true
true
true
Java
Java 21
```

Análisis de cada instrucción:

- **System.out.println(a == b);** → `false`. Imprime false ya que evitamos el Pool al obligar a Java a crear un objeto en el heap con "new".
- **System.out.println(a == c);** → `true`. Imprime true ya que compara a ("Java") con c, este ultimo siendo "Ja" + "va" que es justamente igual a "Java".
- **System.out.println(a == d);** → `false`. Da false, ya que cuando se ejecuta la instrucción d += "va";, la concatenación involucra una variable en lugar de constantes puras, por lo que se resuelve en tiempo de ejecución. Java creó una nueva instancia en el Heap. Aunque el texto final resultante sea "Java", la dirección de memoria de d es distinta a la de a en el String Pool.
- **System.out.println(a.equals(d));** → `true`. El método `.equals()` evalúa la secuencia exacta de caracteres y no la dirección de memoria, por ende da true a diferencia de **Syste.out.println(a == d)**.
- **b = b.concat(" 21");** → Genera una nueva cadena `"Java 21"` en memoria que se asigna a la variable b, por eso **System.out.println(b)** imprime "Java 21".
- **System.out.println(a);** → `Java`. Solo imprime "Java"ya que es el valor de la variable, y esta no sufrió algun cambio

Variables que comparten objeto: Las variables `a` y `c` comparten la misma referencia en el *String Pool*.

Conceptos

String Pool: Es una región especial dentro del Heap gestionada por la JVM para la optimización de memoria. Almacena una única copia de cada String, lo que evita duplicados cuando se crean cadenas idénticas.

Diferencia entre `==` y `equals()`:

`==`: Compara.

`equals()`: Compara el contenido o valor.

Comportamiento de `concat()`: Crea y retorna una nueva instancia de String con el resultado de la concatenación.

02 Operaciones e índices en StringBuilder

Código de la práctica

```
StringBuilder sb = new StringBuilder("ABCDE");
sb.delete(1, 4);
sb.insert(1, "xyz");
sb.reverse();
sb.replace(1, 4, "-");
System.out.println(sb);
System.out.println(sb.length());
System.out.println(sb.capacity());
```

1. Salida y contenido

1. `sb = "ABCDE"`
2. **sb.delete(1, 4):** Elimina los caracteres entre la posición 1 a la 4, quedando solo `"AE"`.
3. `sb.insert(1, "xyz")`: Inserta la cadena `"xyz"` a partir de la posición del índice 1. Quedando `"AxyzE"`.
4. `sb.reverse()`: Invierte el orden de la secuencia completa de caracteres.`"Exyza"`.
5. `sb.replace(1, 4, "-")`: Reemplaza los caracteres entre los índices 1 y 4, correspondientes a `"xyz"`, por el carácter `"-"`, siendo la salida `"E-a"`.

Salida exacta producida por los `System.out.println`:

```
E-a
3
21
```

2. Explicación de índices, `length()` y `capacity()`

- **¿Por qué `delete(1, 4)` no elimina cuatro caracteres?**
  En Java, la manipulación de rangos sobre secuencias se lee así [x, y). Por ende el 4 siempre queda excluido.
- **Diferencia entre `length()` y `capacity()`:**
  - `length()`: Retorna la cantidad de caracteres.
  - `capacity()`: Retorna el tamaño total de la memoria reservada internamente, la cual por defecto es la longitud inicial más 16 (5+16 = 21).

03 Referencias compartidas

Código de la práctica

```
StringBuilder a = new StringBuilder("Hola");
StringBuilder b = a;
StringBuilder c = new StringBuilder("Hola");
a.append(" mundo");
b.delete(0, 5);
System.out.println(a);
System.out.println(b);
System.out.println(c);
System.out.println(a == b);
System.out.println(a.equals(b));
System.out.println(a.equals(c));
```

1. Salidas y explicación de modificaciones

Salida exacta:

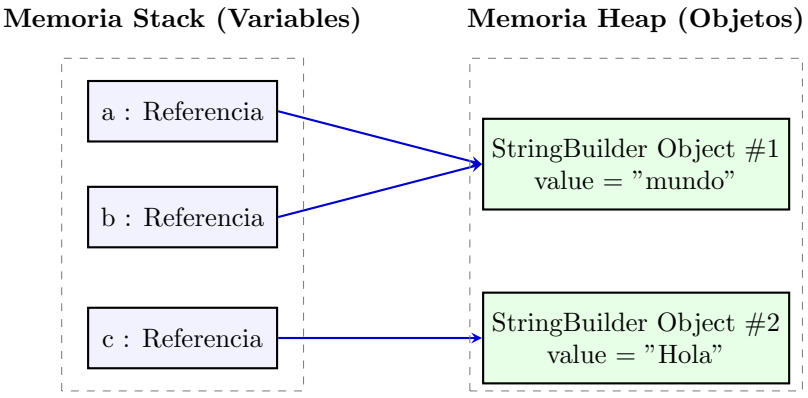
```
mundo
mundo
Hola
true
true
false
```

Análisis de modificaciones:

- `StringBuilder b = a;` asigna la referencia del objeto `a` a la variable `b`. Ambos identificadores apuntan al mismo espacio de memoria en el Heap.
- `a.append(" mundo");` modifica el objeto compartido, cambiando su contenido a `"Hola mundo"`.
- `b.delete(0, 5);` elimina los primeros 5 caracteres (`"Hola "`) sobre el mismo objeto referenciado por `b` y `a`, dejando el buffer con el valor `"mundo"`.
- `System.out.println(a);` e `println(b);` imprimen `mundo` dado que apuntan a la misma entidad.
- `System.out.println(c);` imprime `Hola`, ya que `c` se mantuvo como un objeto independiente en una dirección de memoria distinta.
- `System.out.println(a == b);` retorna `true` porque ambos almacenan exactamente la misma dirección de memoria.
- `System.out.println(a.equals(b));` retorna `true`.

- `System.out.println(a.equals(c));` retorna `false`. La clase `StringBuilder` **no sobreescribe** el método `equals()` de `java.lang.Object`. Por ende, `equals()` en `StringBuilder` realiza una comparación idéntica a `==` (por referencia) y no por contenido textual.

2. Esquema de memoria



Preguntas apartado 10

1. **¿Por qué los atributos del editor deben ser privados?**

   R: Para hacer más robusto el código al encapsular las variables, y que otros métodos no cambien por error la memoria de la variable.

2. **¿Qué diferencia hay entre un método public y uno private?**

   R: Lo accesible que son, Public puede ser alterado en cualquier parte del proyecto, mientras que private solo en la clase en la que está.

3. **Si quitas public de un método, ¿qué acceso tiene y puede llamarlo PruebaEditor desde el mismo paquete?**

   R: Si cambio public por private no tendrá acceso para PruebaEditor y no podrás llamarlo.

4. **¿En qué se diferencia el acceso de paquete de protected?**

   R: El protected es igual que el private, pero añade un permite acceso a clases del mismo paquete y subclases (las cuales pueden estar en diferentes paquetes) por medio de la herencia.